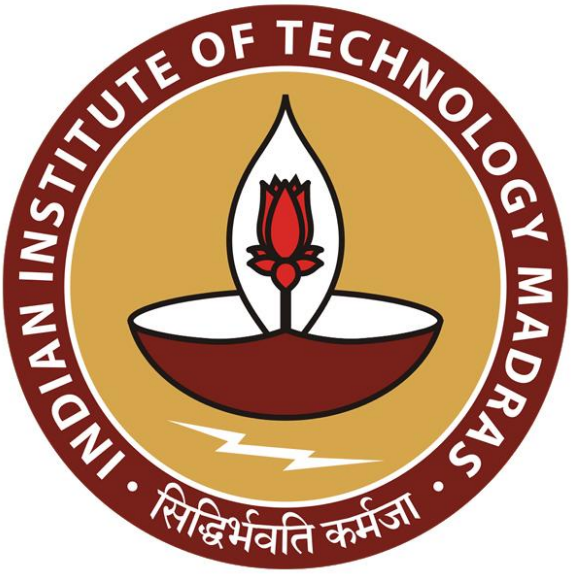


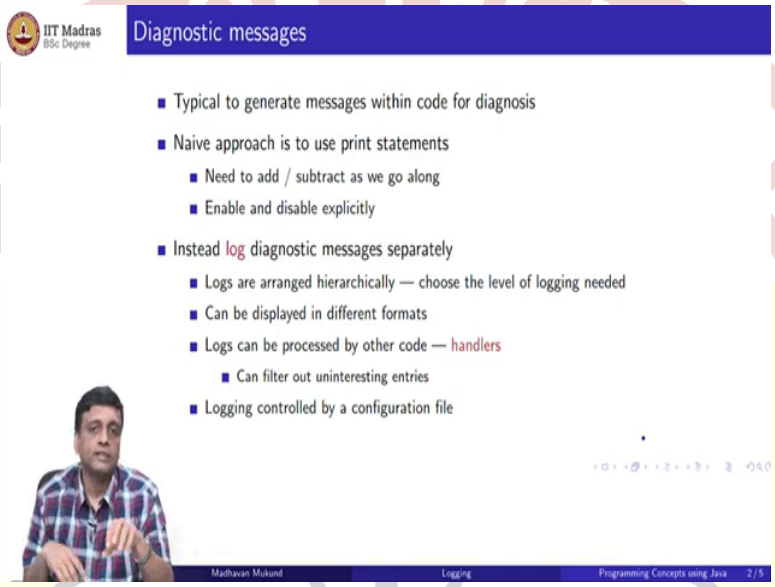
IIT Madras

ONLINE DEGREE

Programming Concepts using Java
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Logging

So, the last item that we will talk about in this theme of errors and exceptions is logging.

(Refer Slide Time: 0:21)



It is quite typical when we write code and something goes wrong to print out a diagnostic saying at this point x is so much. So, we want to know exactly what happened. So, we got an unexpected output, maybe the program did not crash, but you we did not get the answer that we wanted. So, we want to trace through the program and find out where the value that we were expecting was different from what we actually had.

So, you insert these diagnostic messages which are typically inserted as print statements. So, normally, I mean you could have a more sophisticated development environment where you have a debugger where you can halt the execution and inspect. But if you do not use a debugger, then this normal technique is to actually flag it through a print statement.

So, the problem with these print statements is, of course, that depending on where you feel the code is, you have to go and insert a print statement. Once that code is fixed, you have to remove that print statement go wrong on unnecessary print statements cluttering up your

code. So, you have to do this dynamically, or to keep adding and subtracting print statements and print statements basically cut clutter up your code anyway.

At the same time, you cannot dispense with this, you might argue that a debugger, if you use properly, you do not need print statements. But still, when systems are running in a public space, if they are not trivial things, suppose it is a banking system or a railway reservation system or something like that, there is also a need to keep this diagnostic information in case something goes wrong or in case something, somebody queries something, somebody says, Oh, you know, I wanted this information, and you have not captured it properly, then you have to go and say that see when the at the time when you entered information, this is what you entered. And this is what we captured.

So, you might want to keep this diagnostic information also for later accounting or auditing the code. So, the solution to this is to log these messages. So, write a log is something that in real life consists of a register or a physical diary or something where people write down every step of what they do. So, that is where the origin of the word log goes. So, log is a systematic account of what happened. So normally, this is a manual account of every step that happened or every step that you chose to record. It is like a diary.

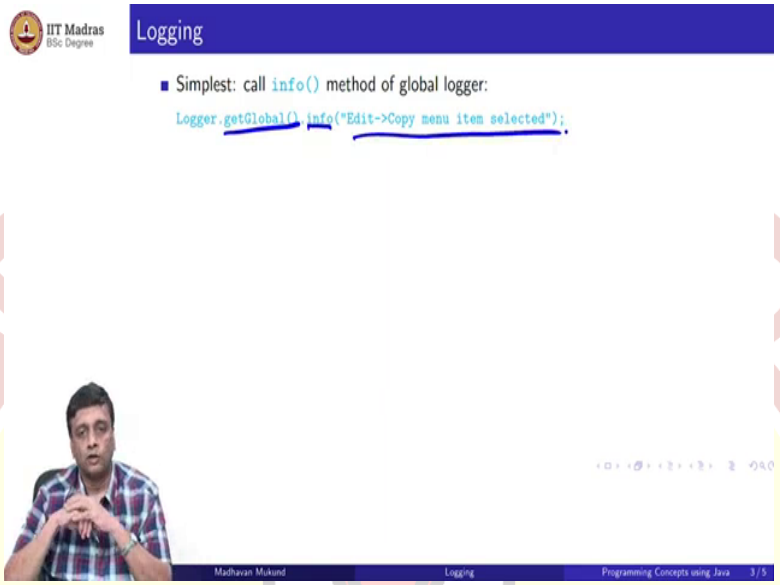
So, in Java, the difference between these diagnostic messages being printed out which are all appearing in some sense at the same level, and a log is that logs are arranged hierarchically. So, some messages are more significant than other messages. And also, some logs are kind of super, or subsume other logs. So, the logs themselves are arranged in a hierarchy. And the messages are also in hierarchy.

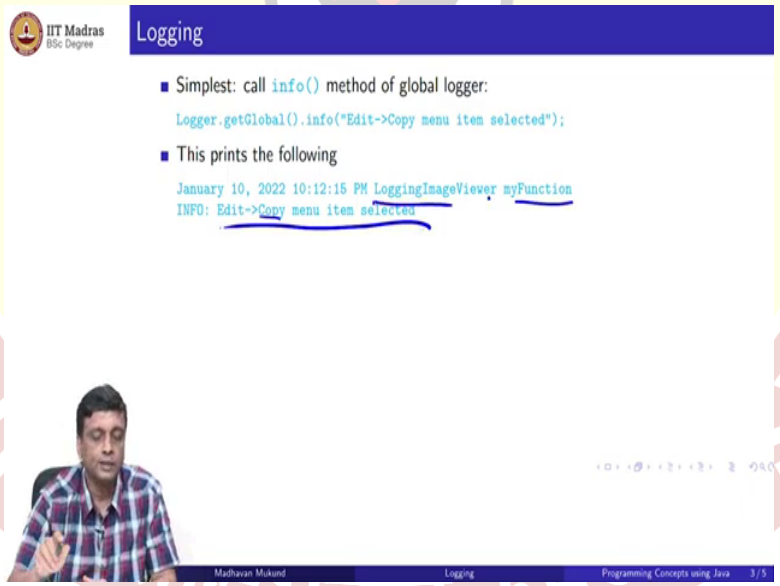
So, once we do this, we kind of separate out the logging part, from the displaying parts earlier, we are kind of hardwired, the fact that we send a string and it gets printed, nothing more, you can do it. Now we have a log, which abstractly stores all this information. And now you can display it however you want. So, that is one advantage.

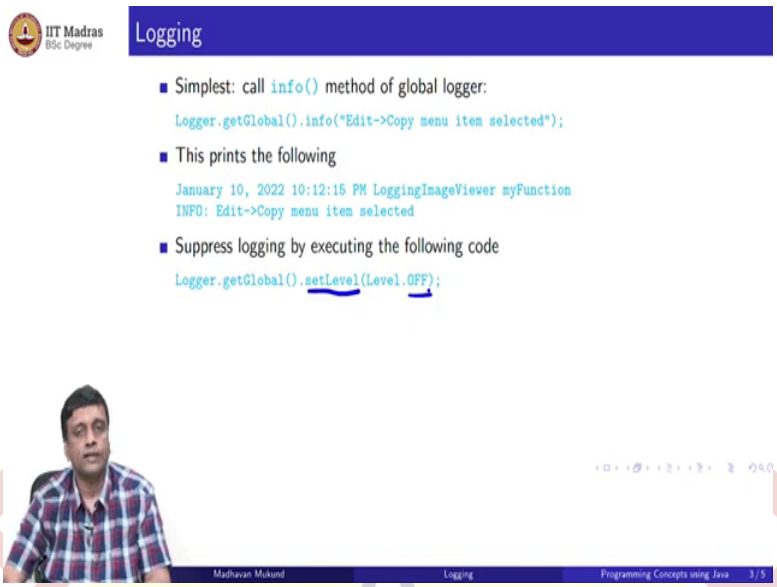
The other thing is that your code does not have processed the log. So, the lag can be processed by some other courts. So, you can have what are called handlers, which can choose to ignore certain things and only print certain things. So, the log can be filtered out. And what we will see is that you can also control this logging from outside the program, it is not something so we solved this problem with the print statement is that when you put it in, it

will remain there. If you do not want the print statement, you have to remove it manually. In principle, that is also true of any such statement. But it turns out that logs can also be controlled from outside without changing the code.

(Refer Slide Time: 3:53)







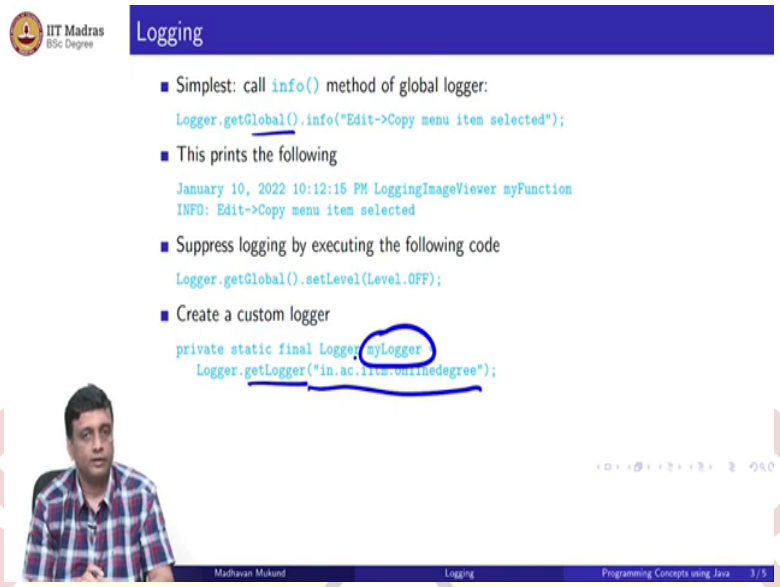
So, here is the simplest way to use a log. So, there is a built-in global logger class. So, you can take that logger class, and you can get a global, individual logger for yourself. So, this is an object which collects these log information's. And you can now use this info method and give it a string. So, this string will now be added to that log. So, remember, a log is unlike a print statement, a log is going to be an accumulation of these things. So, it is actually going to be stored.

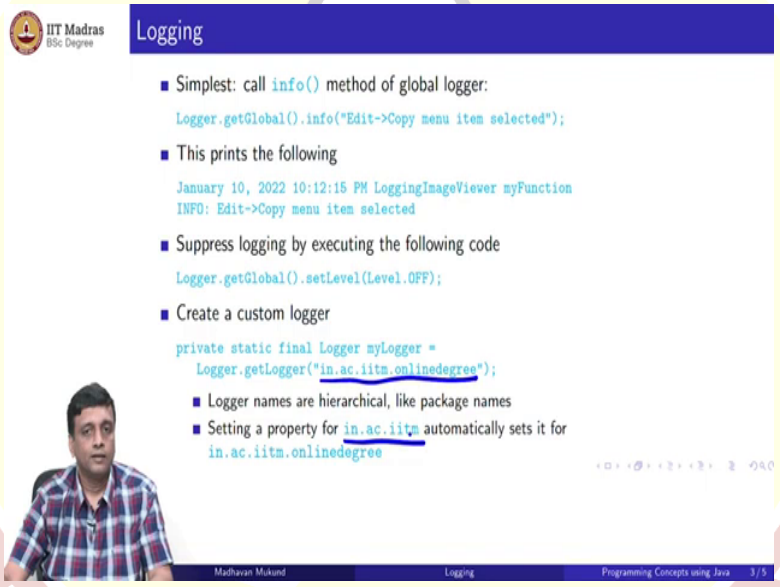
So, this will then print out some diagnostic thing, like the time that it happened and, and what happened and what is the name of the function and the actual message. So, this is the actual message that you put in there. So, each entry that you add to the log automatically also gets extra info, meta data as it is called, like the time date and all this thing which you do not have to put in explicitly. So, that is another advantage.

In a print statement, you do not know when it happened. It just appeared on the screen at some point, it is here. You also get this extra information about when it happened and so on and in what context so this from a particular application and a particular function. So, you can now one way to turn off the logging is to run this command. So, it says, set the logging level to be off, which is do not print any logs.

So, you can now of course, this means that you have to recompile. So, it still involves doing it, but you can do it in one short for all the logs. So, it is not like the print statement, you can suppress all the print statements very easily this way. So, you can at the beginning of your code, see at the top of main, you can decide whether to set the logging level off or on.

(Refer Slide Time: 5:35)



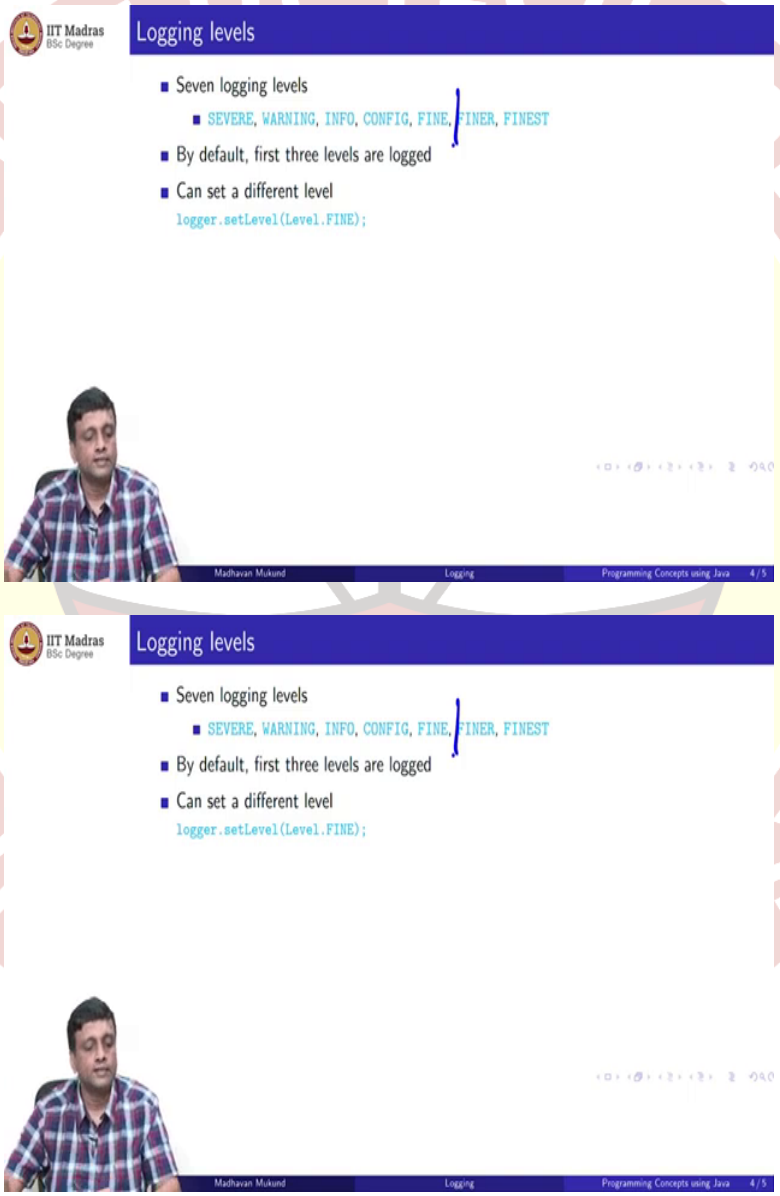


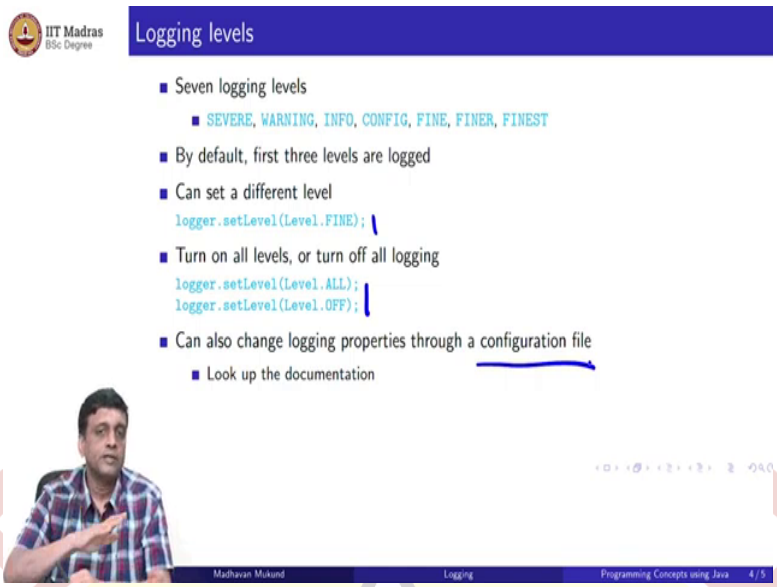
More interestingly, you can create different loggers, you may not want all your log messages to go and sit in the same page, depending on where it came from, you might want to put it in different places. So, this is a bit like the packages that we saw before. So, packages allow you to organize your code in groups.

Similarly, you can log your messages in groups. So, you can create your own logger, so not the getGlobal logger, but you can get a logger of your own and provide it a name. So, this name is like a package name. So, this will now create a specific logger, and it is stored in this variable. So now, you will send the information to that variable. So, this is how you would create a custom logger. And these log names are actually hierarchical.

So, if I look at if I had a log logger, which was called in dot ac dot iitm, then it could be at a higher level that then the longer for in dot iitm dot online degree. So, if I now set some properties for in dot ac dot iitm, for example, as we will see, you can control which types of messages get logged. So, if you control this for the higher level, it will automatically happen for the lower levels. So, this hierarchy is also there implicitly in this naming of these individual logs.

(Refer Slide Time: 6:56)





So, it turns out that these logging levels are at many levels, at 7 levels. So, you have from severe to finest. So, these are the most in some sense, difficult or dangerous problems. And these are some very micro problems, you might not bother about them at some level. So, by default, everything up to the third level is logged. And you can now set a different level of logging, saying I want everything up to here to be logged.

Now, when you log when you add something to it, you can add a message at a given level. So, I am not going to look at the details here. But we saw that you can just add something to the global log, but you can actually say add something to the global log at level warning, add it to the global log at level final and so. And then the code will continue to say this, but whether it actually gets added or not will be controlled by the level that you have set in the setLevel.

So, we said earlier that you could actually turn everything off and that is a particular instance of the setLevel. So, if you set it to be off, it means that all logging is suppressed from now on for that particular logger. If you set it to be all, then every level is there. So, these are two extremes, you want everything or you want nothing. In between, you can say up to which point you want it. So, these are things that you can do with the logging levels.

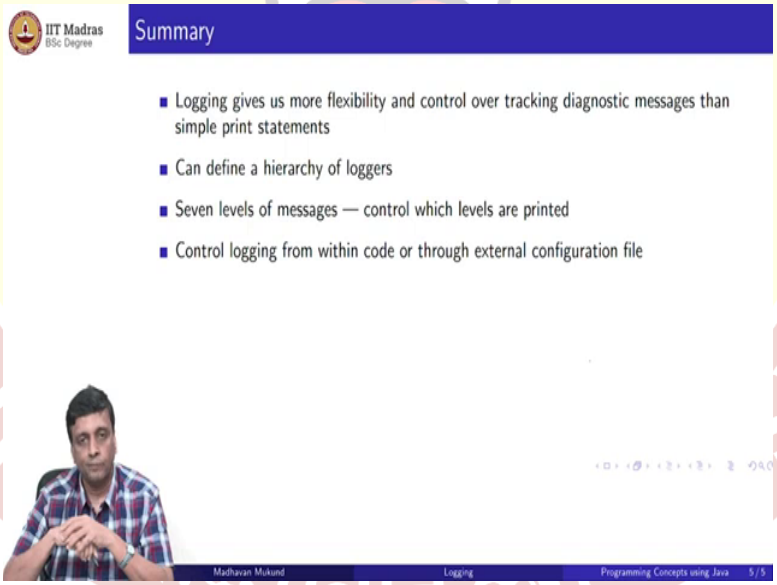
Now, this is all through the code. So, remember that if I want to change the level of logging and this kind of thing, I have to go back, edit the code, recompile it and run it. So, it is not as convenient as you would expect. If this was like something, remember our assert mechanism. So, we put kept all the asserts in the code and then externally, we could say which asserts to

turn on which ones to turn off, you could say turn all of them on, turn on asserts only in this class, turn on asserts only in this package, and so on.

So that was something that happened at runtime, but did not require us to go back and edit the code. Now, if I have these kinds of statements in the code, then if I want to change something about the logging, which logger I want to change, I have to do something. So, it turns out that you can actually do this also outside. But it is not as simple as just passing an argument to Java, you have to actually set up a configuration file.

So, there is a separate file. But the advantage is configuration file is outside your code. So, you can go into this configuration file and update it. And then when you run the code, then the thing will automatically work. So, you do not have to change your code sometimes should do this. So, I am not going to go into details of how to set up a configuration file for loggers, but you can look up the documentation. But the point is that logging is a different mechanism from this print statement for keeping track of diagnostics.

(Refer Slide Time: 9:25)



So, it gives us more flexibility and more control over tracking diagnostic messages than just having a simple print statement. So, we can define this hierarchy of loggers and a hierarchy of levels of messages. So, there are some 7 levels of messages. And we can control which messages are actually going to be logged and which ones are not going to be logged and we can control this logging either by executing a command within our code or by setting up a separate configuration file.